

# 恢复数字序列

## 题目描述

对于一个连续正整数组成的序列，可以将其拼接成一个字符串，再将字符串里的部分字符打乱顺序。如序列8 9 10 11 12，拼接成的字符串为89101112，打乱一部分字符后得到90811211，原来的正整数10就被拆成了0和1。

现给定一个按如上规则得到的打乱字符的字符串，请将其还原成连续正整数序列，并输出序列中最小的数字。

## 输入描述

输入一行，为打乱字符的字符串和正整数序列的长度，两者间用空格分隔，字符串长度 不超过200，正整数不超过1000，保证输入可以还原成唯一序列。

## 输出描述

输出一个数字，为序列中最小的数字。

## 用例

|    |            |
|----|------------|
| 输入 | 19801211 5 |
| 输出 | 8          |
| 说明 | 无          |

## 题目解析

本题“打乱字符的字符串”的是由“连续正整数序列”组成的，而题目又说：正整数不超过1000

因此，本题可以考虑使用 滑动窗口 ，在1~1000整数范围内滑动，滑窗窗口的长度为输入的“连续正整数序列”的长度。

我们只需要统计“打乱字符的字符串”中各字符的数量，以及滑窗中各字符的数量，如果所有字符对于的数量都相同，那么说明对应滑窗就是一个符合要求的连续正整数数列，我们只需要取滑窗第一个数值即可。

滑窗在运动过程中，可以利用差异比较，即当前滑窗相交于上一个滑窗，失去了一个数字，新增了一个数字，当前滑窗只需要针对失去数字和新增数字的字符做数量修改即可。

---

根据考友反馈，本题输入描述中说：正整数不超过1000，但是实际考试时，应该是不超过10000，因此下面代码实现中，将最大正整数改到了10000，大家注意下。

## Java算法源码

```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          String s = sc.next();
8          int k = sc.nextInt();
9
10         System.out.println(solution(s, k));
11     }
12
13     public static int solution(String s, int k) {
14         // 统计乱序字符串中各数字（0~9）的个数
15         int[] count1 = new int[10];
16         for (int i = 0; i < s.length(); i++) {
17             count1[s.charAt(i) - '0']++;
18         }
19
20         // 统计连续正整数组成的序列中各数字（0~9）的个数
21         int[] count2 = new int[10];
22
23         // 滑窗长度固定为k，初始滑窗范围 [1, k]
```

```

24     for (int i = 1; i <= k; i++) {
25         increase(count2, i);
26     }
27     // 若初始滑窗就符合要求, 则返回初始滑窗左边界元素1
28     if (equals(count2, count1)) {
29         return 1;
30     }
31
32     // 后续滑窗范围: [i-k+1, i]
33     for (int i = k + 1; i <= 10000; i++) {
34         increase(count2, i); // 相较于上一个滑窗新增 i
35         decrease(count2, i - k); // 相较于上一个滑窗失去 i - k
36
37         // 若滑窗符合要求, 则返回滑窗左边界 i-k+1
38         if (equals(count2, count1)) {
39             return i - k + 1;
40         }
41     }
42
43     // 若找不到, 则返回-1, 本题说一定会有解, 所以此步走不到
44     return -1;
45 }
46
47 public static boolean equals(int[] a, int[] b) {
48     for (int i = 0; i < a.length; i++) {
49         if (a[i] != b[i]) return false;
50     }
51     return true;
52 }
53
54 public static void increase(int[] count, int num) {
55     while (num > 0) {
56         count[num % 10]++;
57         num /= 10;
58     }
59 }
60

```

```

61 |     public static void decrease(int[] count, int num) {
62 |         while (num > 0) {
63 |             count[num % 10]--;
64 |             num /= 10;
65 |         }
66 |     }
67 | }

```

## JS算法源码

```

1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const [s, k] = (await readline()).split(" ");
7 |     console.log(solution(s, parseInt(k)));
8 | })();
9 |
10 | function solution(s, k) {
11 |     // 统计乱序字符串中各数字（0~9）的个数
12 |     const count1 = new Array(10).fill(0);
13 |     for (let c of s) {
14 |         count1[parseInt(c)]++;
15 |     }
16 |
17 |     // 统计连续正整数组成的序列中各数字（0~9）的个数
18 |     const count2 = new Array(10).fill(0);
19 |
20 |     // 滑窗长度固定为k，初始滑窗范围 [1, k]
21 |     for (let i = 1; i <= k; i++) {
22 |         increase(count2, i);
23 |     }
24 |
25 |     // 若初始滑窗就符合要求，则返回初始滑窗左边界元素1

```

```
26 |   if (equals(count1, count2)) {27 |       return 1;
28 |   }
29 |
30 |   // 后续滑窗范围: [i-k+1, i]
31 |   for (let i = k + 1; i <= 10000; i++) {
32 |       increase(count2, i); // 相较于上一个滑窗新增 i
33 |       decrease(count2, i - k); // 相较于上一个滑窗失去 i - k
34 |
35 |       // 若滑窗符合要求, 则返回滑窗左边界 i-k+1
36 |       if (equals(count1, count2)) {
37 |           return i - k + 1;
38 |       }
39 |   }
40 |
41 |   // 若找不到, 则返回-1, 本题说一定会有解, 所以此步走不到
42 |   return -1;
43 | }
44 |
45 | function increase(count, num) {
46 |     while (num > 0) {
47 |         count[num % 10]++;
48 |         num = parseInt(num / 10);
49 |     }
50 | }
51 |
52 | function decrease(count, num) {
53 |     while (num > 0) {
54 |         count[num % 10]--;
55 |         num = parseInt(num / 10);
56 |     }
57 | }
58 |
59 | function equals(count1, count2) {
60 |     for (let i = 0; i < count1.length; i++) {
61 |         if (count1[i] !== count2[i]) return false;
62 |     }
```

```
63 |     return true;64 | }
```

## Python算法源码

```
1  # 输入获取
2  s, k = input().split()
3  k = int(k)
4
5
6  def increase(count, num):
7      while num > 0:
8          count[num % 10] += 1
9          num //= 10
10
11
12 def decrease(count, num):
13     while num > 0:
14         count[num % 10] -= 1
15         num //= 10
16
17
18 def equals(a, b):
19     for i in range(len(a)):
20         if a[i] != b[i]:
21             return False
22     return True
23
24
25 # 核心代码
26 def solution():
27     # 统计乱序字符串中各数字（0~9）的个数
28     count1 = [0] * 10
29     for c in s:
30         count1[int(c)] += 1
31
```

```

32 | # 统计连续正整数组成的序列中各数字（0~9）的个数
33 | count2 = [0] * 10
34 | # 滑窗长度固定为k，初始滑窗范围 [1, k]
35 | for i in range(1, k + 1):
36 |     increase(count2, i)
37 |
38 | # 若初始滑窗就符合要求，则返回初始滑窗左边界元素1
39 | if equals(count2, count1):
40 |     return 1
41 |
42 | # 后续滑窗范围: [i-k+1, i]
43 | for i in range(k + 1, 10001):
44 |     increase(count2, i) # 相较于上一个滑窗新增 i
45 |     decrease(count2, i - k) # 相较于上一个滑窗失去 i - k
46 |
47 | # 若滑窗符合要求，则返回滑窗左边界 i-k+1
48 | if equals(count2, count1):
49 |     return i - k + 1
50 |
51 | # 若找不到，则返回-1，本题说一定会有解，所以此步走不到
52 | return -1
53 |
54 |
55 | # 输出打印
56 | print(solution())

```

## C算法源码

```

1 | #include <stdio.h>
2 | #include <string.h>
3 | #include <stdbool.h>
4 |
5 | void increase(int count[], int num) {
6 |     while (num > 0) {
7 |         count[num % 10]++;

```

```

8         num /= 10;
9     }
10 }
11
12 void decrease(int count[], int num) {
13     while (num > 0) {
14         count[num % 10]--;
15         num /= 10;
16     }
17 }
18
19 bool equals(const int count1[], const int count2[]) {
20     for (int i = 0; i < 10; i++) {
21         if (count1[i] != count2[i]) return false;
22     }
23     return true;
24 }
25
26 int solution(char *s, int k) {
27     // 统计乱序字符串中各数字 (0~9) 的个数
28     int count1[10] = {0};
29     for (int i = 0; i < strlen(s); i++) {
30         count1[s[i] - '0']++;
31     }
32
33     // 统计连续正整数组成的序列中各数字 (0~9) 的个数
34     int count2[10] = {0};
35
36     // 滑窗长度固定为k, 初始滑窗范围 [1, k]
37     for (int i = 1; i <= k; i++) {
38         increase(count2, i);
39     }
40
41     // 若初始滑窗就符合要求, 则返回初始滑窗左边界元素1
42     if (equals(count2, count1)) {
43         return 1;
44     }

```



```

45 |
46 |     // 后续滑窗范围:  $[i-k+1, i]$ 
47 |     for (int i = k + 1; i <= 10000; i++) {
48 |         increase(count2, i); // 相较于上一个滑窗新增  $i$ 
49 |         decrease(count2, i - k); // 相较于上一个滑窗失去  $i - k$ 
50 |
51 |         // 若滑窗符合要求, 则返回滑窗左边界  $i-k+1$ 
52 |         if (equals(count2, count1)) {
53 |             return i - k + 1;
54 |         }
55 |     }
56 |
57 |     // 若找不到, 则返回-1, 本题说一定会有解, 所以此步走不到
58 |     return -1;
59 | }
60 |
61 | int main() {
62 |     char s[10000];
63 |     int k;
64 |     scanf("%s %d", s, &k);
65 |
66 |     printf("%d", solution(s, k));
67 |
68 |     return 0;
69 | }

```

## C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | void increase(vector<int> &count, int num) {
6 |     while (num > 0) {
7 |         count[num % 10]++;

```

```
8         num /= 10; 9     }
10 }
11
12 void decrease(vector<int> &count, int num) {
13     while (num > 0) {
14         count[num % 10]--;
15         num /= 10;
16     }
17 }
18
19 bool equals(vector<int> &count1, vector<int> &count2) {
20     for (int i = 0; i < count1.size(); i++) {
21         if (count1[i] != count2[i]) return false;
22     }
23     return true;
24 }
25
26 int solution(string &s, int k) {
27     // 统计乱序字符串中各数字（0~9）的个数
28     vector<int> count1(10, 0);
29     for (const auto &c: s) {
30         count1[c - '0']++;
31     }
32
33     // 统计连续正整数组成的序列中各数字（0~9）的个数
34     vector<int> count2(10, 0);
35
36     // 滑动长度固定为k，初始滑动范围 [1, k]
37     for (int i = 1; i <= k; i++) {
38         increase(count2, i);
39     }
40
41     // 若初始滑动就符合要求，则返回初始滑动左边界元素1
42     if (equals(count2, count1)) {
43         return 1;
44     }
45 }
```

```
46 // 后续滑窗范围:  $[i-k+1, i]$ 
47     for (int i = k + 1; i <= 10000; i++) {
48         increase(count2, i); // 相较于上一个滑窗新增 i
49         decrease(count2, i - k); // 相较于上一个滑窗失去  $i - k$ 
50
51         // 若滑窗符合要求, 则返回滑窗左边界  $i-k+1$ 
52         if (equals(count2, count1)) {
53             return i - k + 1;
54         }
55     }
56
57     // 若找不到, 则返回-1, 本题说一定会有解, 所以此步走不到
58     return -1;
59 }
60
61 int main() {
62     string s;
63     int k;
64
65     cin >> s >> k;
66     cout << solution(s, k) << endl;
67
68     return 0;
69 }
```